

Notation

Data. Training set $\{(x_i, y_i)\}_{i=1}^N$, $x_i \in \mathbb{R}^d$. Design $X \in \mathbb{R}^{N \times d}$ has row $x_i^\top$; append 1 to x_i to absorb bias b into w . Linear score $s_i = w^\top x_i + b$. For $y_i \in \{-1, +1\}$, margin $m_i = y_i s_i$.

Prediction. Regression $\hat{y}_i = \theta^\top x_i$. Linear classifier $h(x) = \text{sign}(w^\top x + b)$: $\hat{y} = +1$ if $s > 0$, else -1 . Boundary $s = 0$.

Regression

MSE loss. For $h_\theta(x_i) = \theta^\top x_i$,

J(theta) = 1/N sum (theta^T x_i - y_i)^2 = 1/N ||X theta - y||_2^2.

$\nabla_\theta J = \frac{2}{N} X^\top (X\theta - y)$. Set $\nabla J = 0$: $X^\top X\theta = X^\top y$, so $\theta^* = (X^\top X)^{-1} X^\top y$ if invertible.

Linear regression view. $\hat{y}_i = w^\top x_i$, $L_w(x, y) = \frac{1}{N} \sum_i (y_i - \hat{y}_i)^2$. In covariance notation, $w^* = R_x^{-1} R_{xy}$ when $R_x = X^\top X$ (or scaled covariance/second-moment matrix) and $R_{xy} = X^\top y$.

Ridge / L2. $J_\lambda = \frac{1}{N} \|X\theta - y\|^2 + \lambda \|\theta\|_2^2$. Solution $\theta^* = (X^\top X + N\lambda I)^{-1} X^\top y$. If no $1/N$, use $X^\top X + \lambda I$. Shrinks weights continuously, lowers variance, improves conditioning, usually keeps all features.

Lasso / L1. $J_\lambda = \frac{1}{N} \|X\theta - y\|^2 + \lambda \|\theta\|_1$, $\|\theta\|_1 = \sum_j |\theta_j|$. Promotes sparse weights/feature selection. Not differentiable at 0; subgradient of $|\theta_j|$ is $\text{sign}(\theta_j)$ if nonzero and any value in $[-1, 1]$ at 0. Contrast: ridge shrinks all weights smoothly, lasso can set weights exactly 0.

Covariance matrix. For centered data rows $x_i - \mu$, sample covariance $\Sigma = \frac{1}{N-1} \sum_i (x_i - \mu)(x_i - \mu)^\top = \frac{1}{N-1} X_c^\top X_c$. Entry Σ_{jk} measures how features j, k vary together; correlation normalizes by stds: $\rho_{jk} = \Sigma_{jk} / (\sigma_j \sigma_k)$.

Logistic / MLE

Sigmoid model. $h_w(x) = \sigma(w^\top x + b)$, $\sigma(z) = 1/(1 + e^{-z})$. Identities: $1 - \sigma(z) = \sigma(-z)$, $\sigma'(z) = \sigma(z)(1 - \sigma(z))$. Logistic boundary $h = 0.5 \Leftrightarrow w^\top x + b = 0$.

Bernoulli likelihood. For $y_i \in \{0, 1\}$, $p(y_i | x_i; w) = h_i^{y_i} (1 - h_i)^{1 - y_i}$. Log-likelihood $\ell(w) = \sum_i [y_i \log h_i + (1 - y_i) \log(1 - h_i)]$. MLE $w^* = \arg \max_w \ell(w) = \arg \min_w [-\ell(w)]$.

BCE / negative log-likelihood. $J_{BCE} = -\frac{1}{N} \sum_i [y_i \log h_i + (1 - y_i) \log(1 - h_i)]$. Gradient $\nabla_w J_{BCE} = \frac{1}{N} \sum_i (h_i - y_i) x_i = \frac{1}{N} X^\top (h - y)$. CE is convex for logistic regression in w .

Squared error argmin. Objective $\min_\theta J(\theta)$ with $J(\theta) = \frac{1}{N} \sum_i (h_\theta(x_i) - y_i)^2$.

Linear Classifiers

Bias trick. Replace x by $\tilde{x} = [x; 1]$, w by $\tilde{w} = [w; b]$, so $s = \tilde{w}^\top \tilde{x}$.

Margin constraint. $y_i(w^\top x_i + b) \geq 1$. Equivalently for SVM labels: $w^\top x_i + b \geq 1$ if $y_i = +1$, and $w^\top x_i + b \leq -1$ if $y_i = -1$.

Perceptron. If $y_i w_k^\top x_i \leq 0$, update $w_{k+1} = w_k + \eta y_i x_i$ and $b_{k+1} = b_k + \eta y_i$; otherwise no update. Repeat until all classified correctly or stop. Margin variant keeps updating until $y_i(w^\top x_i + b) \geq 1$.

SVMs

What SVM does. Finds a separating hyperplane with maximum margin, usually by minimizing $\|w\|$ while satisfying class constraints. Decision rule remains $\text{sign}(w^\top x + b)$.

Hard-margin primal. $\min_{w,b} \frac{1}{2} \|w\|^2$ subject to $y_i(w^\top x_i + b) \geq 1$. Margin hyperplanes: $w^\top x + b = \pm 1$. Geometric distance from boundary to either margin plane is $1/\|w\|$; total margin width is $2/\|w\|$. Longer $w \Rightarrow$ smaller margin.

Soft margin / hinge. $J(w, b) = \sum_i \max(0, 1 - y_i(w^\top x_i + b)) + \frac{\lambda}{2} \|w\|^2$. Points with $m_i < 1$ contribute; $m_i \geq 1$ are inactive. Equivalent slack view: allow violations $\xi_i \geq 0$, $y_i s_i \geq 1 - \xi_i$, penalize $\sum_i \xi_i$. Support vectors lie on/inside the margin

and determine the boundary.

Optimization

Gradient descent. $w_{k+1} = w_k - \eta_k \nabla f(w_k)$. Stop when $|f(w_{k+1}) - f(w_k)| < \varepsilon$, $\|\nabla f(w_k)\|$ small, or max iterations reached.

Stochastic gradient descent. GD uses all N examples per update: $\nabla J = \frac{1}{N} \sum_i \nabla \ell_i$. SGD uses one example: $w \leftarrow w - \eta \nabla \ell_i(w)$. Mini-batch uses B : $w \leftarrow w - \eta \frac{1}{|B|} \sum_{i \in B} \nabla \ell_i(w)$. SGD is cheaper/noisier, often faster on large data; full GD is smoother but each step costs more.

Hessian. $H_{ij} = \partial^2 f / \partial w_i \partial w_j$. Quadratic approximation $f(w + \delta) \approx f(w) + \nabla f^\top \delta + \frac{1}{2} \delta^\top H \delta$. At an interior minimum, $\nabla f = 0$; positive definite Hessian is the multi-D analog of second derivative > 0 .

Convexity. If $H(w) \succeq 0$ for all w , f is convex; then any local minimum is global. If $H \succ 0$, locally strictly convex.

Lagrangian / Dual

Primal form. For minimize $f_0(x)$ s.t. $g_i(x) \leq 0$, $h_j(x) = 0$, Lagrangian

L(x, alpha, beta) = f_0(x) + sum_i alpha_i g_i(x) + sum_j beta_j h_j(x), alpha_i >= 0.

Each constraint gets a multiplier.

Dual function. $q(\alpha, \beta) = \inf_x L(x, \alpha, \beta)$. Dual problem: $d^* = \max_{\alpha, \beta: \alpha \geq 0} q(\alpha, \beta)$. Weak duality: $d^* \leq p^*$ for minimization primal optimum p^* . Strong duality can hold under convexity/regularity conditions.

KKT memory. Feasibility, $\alpha_i \geq 0$, stationarity $\nabla_x L = 0$, complementary slackness $\alpha_i g_i(x) = 0$.

Validation

Train / validation / test. Train fits weights; validation chooses hyperparameters/model class; test is untouched until final evaluation. Reusing test for choices leaks information.

K-fold CV. Split training data into K folds. For each candidate, train on $K - 1$ folds and validate on the held-out fold; average validation scores: $CV(\lambda) = \frac{1}{K} \sum_{k=1}^K E_k(\lambda)$. Compared with one split, CV uses data more efficiently and gives a stabler generalization estimate, but costs about K trainings per candidate.

Model selection. Pick hyperparameters by validation/CV, then retrain on all training data with the chosen setting. Report final test performance once. High train error and high val error suggests underfitting; low train error and high val error suggests overfitting.

K-Means

Objective. Given K clusters with centroids $\mu_1, \dots, \mu_K$ and assignments c_i , minimize within-cluster sum of squares $J = \sum_{i=1}^N \|x_i - \mu_{c_i}\|_2^2$.

Algorithm. Initialize centroids. Assignment step: $c_i = \arg \min_k \|x_i - \mu_k\|^2$. Update step: $\mu_k = \frac{1}{|C_k|} \sum_{i: c_i = k} x_i$. Repeat until assignments/objective stop changing. Sensitive to initialization, scaling, and choice of K ; objective decreases each iteration but can reach local minima.

Interpretation. K-means assumes compact roughly spherical clusters under Euclidean distance. Standardize features first if scales differ. Empty cluster fix: reinitialize that centroid or split a large cluster.

Kernels

Gram matrix. Kernel $K(x, z) = \phi(x)^\top \phi(z)$. For data $x_1, \dots, x_N$, $G_{ij} = K(x_i, x_j)$. This is the matrix of pairwise kernel values.

Mercer / PSD test. A valid kernel gives a symmetric PSD Gram matrix for every finite dataset. PSD means $a^\top G a \geq 0$ for all coefficient vectors a . Any negative eigenvalue disproves PSD. Since $\det(G) = \prod_j \lambda_j$, a negative determinant proves mixed eigenvalue signs.

Common kernels. Linear $K(x, z) = x^\top z$. Polynomial $K(x, z) = (x^\top z + c)^d$, $c \geq 0, d \in \mathbb{N}$. RBF $K(x, z) = \exp(-\gamma \|x - z\|^2)$, $\gamma > 0$.

Kernel distance. $\|\phi(x) - \phi(z)\|^2 = K(x, x) + K(z, z) - 2K(x, z)$.

Composition rules. If K_1, K_2 valid: $K_1 + K_2$, cK_1 for $c > 0$, $K_1 K_2$, $f(x)K_1(x, z)f(z)$, integer powers K_1^m , and $\exp(K_1)$ are valid. Schur product theorem: elementwise product of PSD matrices is PSD, explaining product kernels.

Vector Calculus

Shapes. Denominator-layout: derivative of scalar $f \in \mathbb{R}$ w.r.t. vector $w \in \mathbb{R}^d$ is $\nabla_w f \in \mathbb{R}^{d \times 1}$.

Linear terms. $\nabla_w (w^\top x) = x$, $\nabla_x (x^\top y) = y$, $\nabla_x (y^\top x) = y$.

Quadratics. $\|x\|_2^2 = x^\top x$, $\nabla_x \|x\|_2^2 = 2x$. $\nabla_w (w^\top A w) = (A + A^\top)w$; if $A = A^\top$, $2Aw$.

Chain rule. $\nabla_w f(g(w)) = f'(g(w)) \nabla_w g(w)$. Example: $\nabla_w \sigma(w^\top x) = \sigma(w^\top x)(1 - \sigma(w^\top x))x$.

Affine least squares. For $r = Xw - y$, $\nabla_w \frac{1}{2} \|r\|^2 = X^\top r$; without $\frac{1}{2}$, multiply by 2. With ridge $\frac{\lambda}{2} \|w\|^2$, add λw .

Trace tricks. $\nabla_x (a^\top x) = a$, $\nabla_X \text{tr}(A^\top X) = A$, $\nabla_X \frac{1}{2} \|AX - B\|_F^2 = A^\top (AX - B)$. Helpful because many matrix objectives can be rewritten with traces.

Linear Algebra

Transpose. $(AB)^\top = B^\top A^\top$. Scalar transpose equals itself: $x^\top A y = (x^\top A y)^\top = y^\top A^\top x$. If A symmetric, $x^\top A y = y^\top A x$.

Inverse / rank. $(AB)^{-1} = B^{-1} A^{-1}$ when invertible. A^{-1} exists iff A is square and full rank. $X^\top X$ is invertible iff columns of X are linearly independent; otherwise use regularization or pseudoinverse.

Lp norms. $\|x\|_p = (\sum_j |x_j|^p)^{1/p}$. $\|x\|_2 = \sqrt{x^\top x}$; $\|x\|_1 = \sum_j |x_j|$; $\|x\|_\infty = \max_j |x_j|$.

Projection. Projection of y onto $\text{col}(X)$: $\hat{y} = X(X^\top X)^{-1} X^\top y = Hy$. H is symmetric/idempotent: $H^\top = H$, $H^2 = H$. Residual $y - \hat{y}$ is orthogonal to columns of X : $X^\top (y - Xw^*) = 0$.

PSD / PD. For symmetric A , PSD $A \succeq 0$ means $z^\top A z \geq 0$ for all z , all eigenvalues ≥ 0 . PD $A \succ 0$ means $z^\top A z > 0$ for nonzero z , all eigenvalues > 0 . Negative eigenvalue $\Rightarrow$ not PSD. For 2×2 , determinant $< 0 \Rightarrow$ mixed eigenvalue signs.

Eigen facts. Symmetric $A = Q\Lambda Q^\top$ with orthonormal eigenvectors. $A \succeq 0 \Rightarrow x^\top A x = \sum_j \lambda_j (q_j^\top x)^2 \geq 0$. Trace is sum of eigenvalues; determinant is product of eigenvalues.